Language Semantics Driven Design and Formal Analysis for Distributed Cyber-Physical Systems

[Extended Abstract]

Ritwika Ghosh
University of Illinois at Urbana-Champaign
rghosh9@illinois.edu

Sasa Misailovic
University of Illinois at Urbana-Champaign
misailo@illinois.edu

Sayan Mitra
University of Illinois at Urbana-Champaign
mitras@illinois.edu

ABSTRACT

Typically platform specific programming models for distributed Cyber-Physical Systems provide limited portability, code reuse, rigorous verification, and synthesis. *Koord* is a new distributed CPS programming model and language, which treats the platform-independent decision and coordination tasks as a separate concern from platform-dependent concerns of low-level sensing, communication, and control. *Koord* features event-based programming, shared variables for convenient coordination across multiple agents, and interfaces defined by sensor and actuator ports for the (platform independent) application program to interface with program's environment consisting of controllers and the physical plant. We implement the executable semantics of *Koord* in the $\mathbb{K}$ framework. The *Koord* framework also includes formal analysis tools on top of the semantics to enable semantics driven-verification.

CCS CONCEPTS

• **Theory of computation → Formal languages and automata theory**; *Distributed computing models*; • **Computer systems organization → Embedded and cyber-physical systems**; • **Software and its engineering → Embedded software**;

ACM Reference Format:
Ritwika Ghosh, Sasa Misailovic, and Sayan Mitra. 2018. Language Semantics Driven Design and Formal Analysis for Distributed Cyber-Physical Systems: [Extended Abstract]. In *ApPLIED '18: Advanced tools, programming languages, and PLatforms for Implementing and Evaluating algorithms for Distributed systems, July 27, 2018, Egham, United Kingdom.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3231104.3231958

1 INTRODUCTION

Programming systems interacting with physical processes are gaining importance for robotics, manufacturing, IoT, and other cyber-physical systems. Traditional domain specific languages models for these systems are platform specific as they combine various low-level sensing, communication, and control tasks with the higher-level applications [14]. For sophisticated high-level software applications, the tight-coupling of application design with platform-specific details hinders development, portability, code reuse, verification, and synthesis.

Our goal is to develop a programming model for the platform-independent parts of the application while providing a convenient interface to the platform dependent components. Currently, a lack of portability and code reusability hampers developer productivity. There is also a high barrier-to-entry due to the prerequisite of knowledge of platform-specific details for application development. The design choices for a convenient interface between the hardware dependent and independent parts here include three central concepts. First, the separation of discrete and dynamic components of the system, second, the timing issues with message passing protocols used by the agents in the system and third, the level of abstraction needed to design an expressive yet straightforward interface between the platform dependent and independent components. Our system addresses these concerns, thereby democratizing application programming for cyber-physical systems. Also, platform-dependent parts often lack formal models—specifications of low-level controllers and sensors are not available from manufacturers, physical processes can have complicated dynamics. This separation can help combine formal analysis techniques for the application program with model-free and data-driven approaches that apply to the platform-dependent parts of the system.

2 AN OVERVIEW OF KOORD

We give a quick overview of the *Koord* language and its features with an example. For a more detailed presentation of the language, we refer the interested readers to [1]. Each agent runs a copy of the same program.

Figure 1 shows a *Koord* program for a set of five robots to form an equally spaced line. Each agent program has access to two constants a) a unique integer identifier *pid* for itself and b) a list *ID* of identifiers of all participating agents.

Program-Environment Interactions. The agent program can write to actuator ports, whose values are used by the controller to drive the underlying physical plant. The sensor ports reflect the changes in the plant state. The *Koord* semantics allows the program to use a black-box model of the environment. It takes as input the values at the actuator port and provides as output a sampling of values at the sensor ports over an observed time interval. *Lineform* uses a controller called *Motion* which attempts to drive the agent to a target way-point as specified by the value set at actuator port

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ApPLIED '18, July 27, 2018, Egham, United Kingdom
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-5775-3/18/07.
https://doi.org/10.1145/3231104.3231958

Motion.target. *Motion* provides two sensor ports: (i) *Motion.pos*: agent's position in a fixed coordinate system (for example, from GPS), (ii) *Motion.status*: a flag indicating whether the *Motion* controller is active, idle, or failed.

The implementation of a controller such as this may involve path planners, and platform-specific steering and throttle regulators, drivers for specific positioning systems.

```
1  Agent Lineform(i)
2
3  using module Motion:
4    actuators:
5      pos target
6    sensors:
7      pos mypos
8      bool status
9
10 allread:
11   pos x
12
13 update:
14   pre (true);
15   eff:
16     x[pid] = Motion.mypos
17     if ¬ (pid == 1 or pid == 5)
18     Motion.target =
19       (x[pid + 1] + x[pid − 1])/2
```

Figure 1: Line formation program written in *Koord*

Koord *program features.* In a *Koord* system, distributed shared variables are implemented by the *Koord* runtime system and are convenient for coordination across multiple agents. *Lineform* uses a shared array x of type *Position*. It is declared as *allread*, which means that the size of $x[]$ is ID, all agents can read from $x[i]$, but only agent with $pid = i$ can only write to $x[i]$. An *allwrite* variable, gives read/write access to all agents. In the *Koord* semantics, each agent i maintains a *local copy* v_i of each shared variable v. Reads from v get the value of the local copy v_i, writes to v update its value directly. Thus, writes to shared variables are visible to other agents in the next round.

Event-driven programming. Events define how program variables are updated. *Lineform* uses a single *Update* event, which sets the *target* of each agent (except the extremal agents 1 and 5) to be the center of the position of its neighbors. This has a *precondition* which is *true*, and therefore, is always enabled. *Koord* runtime system implements a (partially) synchronous round-by-round execution of events for all agents. That is, for a given execution parameter $\delta > 0$, one event per agent can occur every δ time.

This type of synchronous update rule of Line 19 is a typical example of a large family of textbook algorithms for distributed consensus, rendezvous, optimization, flocking, and pattern formation [2, 11, 17].

Applications. Some examples of applications that have been implemented using *Koord* include :

- **Fischer's Mutual Exclusion Protocol**: This is a well-known timing-based distributed mutual exclusion protocol which uses a shared resource to implement the mutual exclusion [9].
- **HVAC**: This application models a room with a heater, which is on or off based on readings from a thermostat. It is a popular benchmark in hybrid systems literature [7]. The thermal dynamics with the effect of the heater is modeled as the environment of *Koord* program.
- **Lineform**: This is a classic consensus-type algorithm, where agents form a line by navigating to the midpoint of their neighbors [11]. The conventional control theoretic analysis ignores issues of concurrency, detailed dynamics, unlike our implementation.
- **SATS landing protocol**: This is a simplified version of distributed landing protocol originally designed for small aircraftsl [8, 12], but we apply it to drones while ensuring that two consecutive drones maintain the required separation between them.
- **Traffic**: In this program, the system handles agents navigating a traffic intersection without traffic lights, by communicating through shared variables. The 4-way intersection has 4 critical sections, and an agent looking to make a turn at the intersection needs at most three of the said critical sections, and one going straight through needs at most two of them.

Lineform has the same structure as commonly used protocols in distributed robotics. Modifying the *target* computation logic gives rise to a variety of other platooning, formation, flocking, and consensus-type protocols. Fischer's protocol is a standard timing-based mutual exclusion algorithm. Traffic shows how shared memory can be used for physical coordination. The SATS protocol is an existing benchmark explored in is a simplified version of a real protocol developed by NASA.

3 KOORD SEMANTICS

We have implemented a formal executable semantics of *Koord* in the $\mathbb{K}$ framework [16].

An agent program consists of i) declarations of the interface between the programs and the controller module, ii) declarations of program variables and iii) events, consisting of preconditions and effects. Agent programs can import a controller module. The module import specifies the interface of the controllers: all input and output ports for actuators and sensors that the program uses. The program can also contain shared and local declarations. The program also includes events which include a boolean **pre**condition, and an **eff**ect. The effect of an event may include the standard control flow constructs including statements, conditional statements, as well as atomic blocks of statements, function calls and assignment statements. For a full syntax, the reader is referred to [1].

3.1 Configurations

The semantics of a system of agents executing *Koord* programs in our framework are described as a nondeterministic, timed automaton [9]. We define the state of the automaton using *configurations* that include the state of the individual agents, as well as specific global information.

(i) *Program transitions* correspond to agent program events executing. Each such transition may involve reading sensor ports, performing computations using local and shared variables, and writing to actuator ports. Multiple agent events interleave in arbitrary order. Program transitions take zero logical time. (ii) *Environment transitions* correspond to δ amount of time elapsing, where $\delta > 0$ is a constant *sampling period*[1] parameter. During this time, the program variables and the actuator port values remain constant, while the state of the environment (controller and plant) and the values of

[1] Usually determined either experimentally to capture observable changes in the sensors.

the sensor ports change according to the dynamics of the environment. If a detailed model of the environment is not available, our formalization only requires an executable function. This function takes as input the values of the actuator ports, and provides as output the changes in the plant seen by the sensors after δ time. The user can provide a time t_{max}, for the execution of the system. The number of environment transitions observed by the *Koord* program is therefore, $\lfloor \frac{t_{max}}{\delta} \rfloor$.

System Configurations. A *system configuration* with sampling parameter δ is a tuple $C_\delta = (L, S, \tau, turn)$: (i) $L = \{L_i\}_{i \in ID}$ is a list of *agent configurations*–one for each participating agent. (ii) $S : Var \mapsto Val$ is the *global context*, mapping all shared variable names to their values. (iii) $\tau \in \mathbb{R}^+$ is the *global time*. (iv) $turn \in \{\texttt{prog}, \texttt{env}\}$ is a binary variable determining whether program or environment transitions are being processed. Each program transition corresponds to the execution of events by an agent.

Agent Configurations. The configuration of an agent contains its program code, its *local context* mapping all variables in its program to their values, the mapping of actuator and control variables to values, and the next transition for the agent (see [1]).

The agent semantics are specified using rewrite rules; which drive the event-level semantics, which in turn are used to define the global automaton. As an example of a rewrite rule, we present an expression evaluation rule for addition. The execution first evaluates the left subexpression, then the right subexpression given that left is already evaluated to a value and finally adding the two values using the rule provided below.

$$\frac{v_1 + v_2 \rightarrow_E v_3}{\langle S, L_i, v_1 + v_2 \rangle \rightarrow_E \langle S, L_i, v_3 \rangle}$$

Statement rewrite rules are used to define event executions followed by an environment transition. The semantics ensures that starting from a global configuration, each agent processes its program using a sequence of statement rewrites, until its event is executed, and its *turn* set to env at the end of the event execution. After that, the global context S is copied into local context of each agent i ($L_i.M$), thus ensuring that all agents have the latest shared variable values before the next program transition. In an actual execution, the controller would run the program on hardware, whose sensor ports evolve for δ time between program transitions.

Implementation. The program is first parsed using a standard indentation parser implemented in python. We used $\mathbb{K}$, which a rewriting-based executable framework for defining language semantics to implement the executable semantics for *Koord*. One can view a language semantics naturally as a set of reduction rules over configurations. Components of a configuration are called *cells* in $\mathbb{K}$. In practice, the configuration in $\mathbb{K}$ includes several bookkeeping elements.

Configurations organize the components in elements called *cells*. Cells are labelled, have types indicating their intended content, and help specify the rewrite rule in context. $\mathbb{K}$ allows underspecification of rewrite rules, meaning, only the rewrite rules affecting part of the configurations need to be specified if the rule doesn't affect the other parts of the rule. Our notion of configurations translate in a straightforward manner to $\mathbb{K}$ configurations. We refer the interested reader to [1] for detailed discussions on semantics and implementation.

4 FORMAL ANALYSIS SUPPORT

We support two verification methods in our framework: inductive invariant checking and bounded model checking. Both methods rely on reachability analysis performed using the executable semantics of *Koord* implemented using $\mathbb{K}$.

Verification of inductive invariants requires the following: a) The user determines a candidate inductive invariant. b) The user provides an explicit closed-form model of the environment.In many cases, the environment may not be available as a closed form function. The reachability component involving dynamic behavior becomes an issue, in that case.For nonlinear dynamical systems, computing the reachability for dynamic behaviors or trajectories is usually undecidable. Moreover, if the environment is a black-box, then we can only hope to derive statistical guarantees about said behaviors.

KoordBMC is a tool for checking bounded invariants (n-invariants) for *Koord* applications using explicit state reachability analysis. It works with both black-box and explicit environment. The input to *KoordBMC* is (i) P: the program, (ii) inv: a candidate invariant function, (iii) ID: set of agents, (iv) δ: time step size and (v) n: length executions to check.

KoordBMC uses the *Koord* executable semantics to produce all reachable configurations during program transitions. The black-box controllers generate traces of the sensor ports of the system during the environment transitions of duration δ. These traces only give a sampling of values at the sensor ports and not all the intermediate values that the system reaches. *KoordBMC* fills these gaps using the DryVR [6] tool which over-approximates the dynamic behavior of a systems for a broad class of white-box and black-box environment models by learning the sensitivity of the behavior with respect to the configurations. DryVR gives a probabilistic guarantee about the correctness of the learned sensitivity function[2], and assuming that the sensitivity is correct it soundly over-approximates the dynamic behavior of the system.

The tool outputs 'safe' if the inv is indeed an n-invariant, 'unsafe' if it finds a counter-example execution of length n, or 'unknown' if the sensitivity analysis is inconclusive.

These tools have been used to prove invariants about the example applications in presented in Section 2, for instance in the Fischer's protocol the desired invariant is that no two processes are in the critical section simultaneously. This is not an inductive invariant, so we used bounded model checking as well as a sequence of inductive invariants that act as a proof for the mutual exclusion property. Similarly, for *Lineform*, the desired invariant is that no two agents collide, while remaining in the convex hull of their initial positions. While we require the user to provide a candidate inductive invariant for the inductive invariant checker component, the bounded model checking tool makes the verification procedure accesible to the programmer without expertise in formal methods.

Bounded model checking is hampered by the state space explosion problem, as expected. However, we have several heuristics for pruning the search tree when the program has certain properties.

[2]The paper shows that 20-40 traces are enough to learn sensitivity of common nonlinear systems with nearly 100% accuracy.

For instance, if the events do not write to any shared variables, it is enough for *KoordBMC* to explore only one order of execution as opposed to a worst case of $n!$ orders where n is the number of agents in the system. Domain knowledge and observing symmetries in the program can also lead to huge improvements in performance.

The importance of using programming language semantics in this setting is to verify the *implementation* of these protocols in a programming language, as opposed to a formal description of a (hybrid) automaton. Traditional theorem provers and model checking tools require the encoding of every *automaton* with states and transitions in the tool. In contrast, our tool makes these formal analysis tools available to the programmer without requiring expertise in formal methods.

5 HARDWARE DEPLOYMENT

Running *Koord* programs on actual hardware is simplified by our system design.The software stack that *Koord* employs for hardware deployment uses the StarL robotics framework [18] as middleware between the *Koord* front-end and the physical platforms, which are running ROS (Robot Operating System) Indigo [15]. StarL has been implemented in Java, a design choice based on the existence of several libraries used to simplify the intermediate code. The *Koord* front-end uses a python based parser to generate the corresponding Java Application that runs on the physical agents. We assume that the participating agents run clock synchronization algorithms, and with negligible drift as compared to δ, we can ensure that the controller only starts driving the physical plant after the maximum possible time to execute a program transition step for each agent has elapsed. Shared memory is implemented in our system using message passing over WiFi. We use the VICON positioning system as the position sensor for the *Motion* controller, which for our experiments also translates to the fact that all the robots are connected to the same WiFi router. We chose UDP as a messaging protocol for implementation of shared memory semantics, considering that on the same network, there is very low chance of memory loss. In fact, in our experiments we found that no messages were dropped as long as the WiFi router maintained connectivity. Currently, we assume that the system knows all the participating robots beforehand. It is only required by the verification procedures we support, not the language semantics. We have developed our own software to do participant discovery for the *Koord* system.

6 RELATED WORK

There are many DSL that aim to automate different development stages for robotic systems. In [13], the authors survey 41 representative languages and categorize them according to the subdomain of robotics (e.g., robot structure, kinematics, planning and control, coordination, etc.) that the DSLs target. Most of these languages are proprietary or generate executables that are tied to specific platforms.

There is extensive work on programming distributed computing systems using the shared memory paradigm and the related consistency models [3, 4, 10]. *Koord* provides distributed shared variable abstraction for programming multiple agents. The consistency semantics implemented here is that the writes to shared variables are propagated to all the agents, and become visible to other agents reading the variable after one round (δ time units). In the fault-free synchronous model considered here, any number of gossip-based algorithms can be used to implement this semantics in the *Koord* runtime system.

The P language [5] is an asynchronous programming language designed specifically for distributed systems to eliminate certain types of concurrency bugs. The interacting state machines in *P* communicate using shared events instead of shared variables. *Koord* programs interact with the physical environment through the sensor and actuator ports, which is a fundamental aspect of the language and its semantics. The cyber-physical interactions are not supported in P.

REFERENCES

[1] . A design and analysis framework for distributed CPS. https://cyphyhouse.github.io/papers/koord.pdf. Accessed: 2018.

[2] V.D. Blondel, J.M. Hendrickx, A. Olshevsky, and J.N. Tsitsiklis. 2005. Convergence in multiagent coordination consensus and flocking. In *Proceedings of the Joint forty-fourth IEEE Conference on Decision and Control and European Control Conference*. 2996–3000.

[3] Brad Calder, Ju Wang, Aaron Ogus, Niranjan Nilakantan, Arild Skjolsvold, Sam McKelvie, Yikang Xu, Shashwat Srivastav, Jiesheng Wu, Huseyin Simitci, Jaidev Haridas, Chakravarthy Uddaraju, Hemal Khatri, Andrew Edwards, Vaman Bedekar, Shane Mainali, Rafay Abbasi, Arpit Agarwal, Mian Fahim ul Haq, Muhammad Ikram ul Haq, Deepali Bhardwaj, Sowmya Dayanand, Anitha Adusumilli, Marvin McNett, Sriram Sankaran, Kavitha Manivannan, and Leonidas Rigas. 2011. Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency. In *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles (SOSP '11)*. ACM, New York, NY, USA, 143–157. https://doi.org/10.1145/2043556.2043571

[4] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. 2007. Dynamo: Amazon's Highly Available Key-value Store. In *Proceedings of Twenty-first ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. ACM, New York, NY, USA, 205–220. https://doi.org/10.1145/1294261.1294281

[5] Ankush Desai, Vivek Gupta, Ethan Jackson, Shaz Qadeer, Sriram Rajamani, and Damien Zufferey. 2013. P: Safe Asynchronous Event-driven Programming. *SIGPLAN Not.* 48, 6 (June 2013), 321–332. https://doi.org/10.1145/2499370.2462184

[6] Chuchu Fan, Bolun Qi, Sayan Mitra, and Mahesh Viswanathan. 2017. DryVR: Data-driven verification and compositional reasoning for automotive systems. In *Computer Aided Verification (CAV)*.

[7] Ansgar Fehnker and Franjo Ivancic. 2004. Benchmarks for Hybrid Systems Verification. In *HSCC (Lecture Notes in Computer Science)*, Rajeev Alur and George J. Pappas (Eds.), Vol. 2993. Springer, 326–341.

[8] Taylor Johnson and Sayan Mitra. 2012. A Small Model Theorem for Rectangular Hybrid Automata Networks.

[9] Dilsun K. Kaynar, Nancy Lynch, Roberto Segala, and Frits Vaandrager. 2005. *The Theory of Timed I/O Automata*. Morgan Claypool.

[10] Avinash Lakshman and Prashant Malik. 2010. Cassandra: A Decentralized Structured Storage System. *SIGOPS Oper. Syst. Rev.* 44, 2 (April 2010), 35–40. https://doi.org/10.1145/1773912.1773922

[11] M. Mesbahi and Magnus Egerstedt. 2010. *Graph-theoretic Methods in Multiagent Networks*. Princeton University Press.

[12] César Muñoz, Víctor Carreño, and Gilles Dowek. 2006. *Formal Analysis of the Operational Concept for the Small Aircraft Transportation System*. Springer Berlin Heidelberg, Berlin, Heidelberg, 306–325. https://doi.org/10.1007/11916246_16

[13] Arne Nordmann, Nico Hochgeschwender, and Sebastian Wrede. 2014. *A Survey on Domain-Specific Languages in Robotics*. Springer International Publishing, Cham, 195–206. https://doi.org/10.1007/978-3-319-11900-7_17

[14] Arne Nordmann and Nico Hochgeschwender Sebastian Wrede. 2014. Robotics DSL Zoo. (2014).

[15] Morgan Quigley, Ken Conley, Brian Gerkey, Josh Faust, Tully Foote, Jeremy Leibs, Rob Wheeler, and Andrew Y Ng. 2009. ROS: an open-source Robot Operating System. In *ICRA workshop on open source software*, Vol. 3. Kobe, Japan, 5.

[16] Grigore Rosu and Traian Florin Serbanuta. 2014. K Overview and SIMPLE Case Study. In *Proceedings of International K Workshop (K'11) (ENTCS)*, Vol. 304. Elsevier, 3–56. https://doi.org/10.1016/j.entcs.2014.05.002

[17] John N. Tsitsiklis. 1987. On the stability of asynchronous iterative processes. *Theory of Computing Systems* 20, 1 (December 1987), 137–153.

[18] Adam Zimmerman and Sayan Mitra. 2012. A Programming Environment for Peer-to-Peer Applications over Ad hoc WiFi and Android phones.